

Seam Playbook v1

Purpose

This playbook records the seam classes found during the Jarvis seam-hunting session, the proof patterns that exposed them, the boundary laws that close them, and the regression expectations that keep them closed.

The goal is not to preserve random bugs. The goal is to preserve repeatable failure families, so future debugging starts from architecture law instead of starting from confusion.

Core Doctrine

Method

1. Feel the inconsistency.
2. Pressure the system until it reveals the failure shape.
3. Name the seam class.
4. Identify the owning boundary.
5. Patch at the boundary, not inside the symptom.
6. Add regression tests for the human path, not only the happy path.
7. Verify the visible artifact, not just the internal claim.

Engineering Law

A seam is closed only when:

- the owning boundary is identified,
 - the failure class is blocked at that boundary,
 - regression coverage proves the failure cannot silently return,
 - and the user-visible artifact matches the intended finalized output.
-

Boundary Map

1. Intake / Admission Gate
2. Intent Binding / OTEM Task Canonicalization
3. Prompt Assembly / Context Integrity
4. Generation Stability
5. Output Finalization
6. Streaming / Delivery

Seam Entries

1. Output Completion Seam

Class

Visible output completion failure.

Signature

- response ends mid-sentence
- no closure marker
- no explicit truncation notice
- visible artifact looks hard-cut rather than intentionally brief

Owning Boundary

Generation -> Output Finalization

Root Cause

Replies were allowed to pass through identity/safety finalization without a structural completion law. Streaming paths could also expose broken fragments before final output repair.

Boundary Law

Every visible reply must terminate in structurally complete form, or be explicitly marked as truncated.

Fix Shape

- add completion guard at finalization boundary
- trim to last safe boundary
- emit visible truncation notice when budget pressure breaks the ending
- route streamed output through the same finalization boundary

Regression Expectations

- clipped fragments are repaired or visibly marked
- clean completions remain unchanged
- stream path cannot leak incomplete endings before guard runs

2. Repetition Collapse Seam

Class

Generation instability / repetition loop leak.

Signature

- repeated 3-5 token sequences
- phrase-frequency spikes in the tail
- low-entropy looping endings
- no semantic progression

Owning Boundary

Generation -> Output Finalization

Root Cause

The model could enter repetition-collapse tails under pressure, and visible output had no loop guard.

Boundary Law

No visible reply may expose uncontrolled repetition-loop tails.

Fix Shape

- detect repeated token sequences
- detect phrase-density spikes in the tail
- detect low-entropy loop tails
- trim to last safe boundary
- emit visible notice: response truncated due to repetition loop
- record repetition_detected in completion trace

Regression Expectations

- direct loop cases are trimmed
- soft loop tails are caught conservatively
- normal rhetorical repetition is not over-trimmed

3. Scaffold Leakage Seam

Class

Internal instruction / scaffold leakage into visible output.

Signature

Visible output contains internal orchestration labels such as:

- mission_critic_next
- specialist_behavior
- planner or routing language
- meta-instruction framing that belongs to internal control layers

Owning Boundary

Prompt Assembly / Visible Output Isolation

Root Cause

Internal scaffolding and orchestration text became renderable in the same output space as user-facing answers.

Boundary Law

Internal planner, critic, routing, and specialist directives must never be renderable in the user-visible answer channel.

Fix Shape

- hard-separate internal and visible instruction layers
- strip or block internal directive patterns from visible output
- prevent scaffold fragments from re-entering future context windows

Regression Expectations

- no internal labels reach user-visible output
- assistant/planner fragments do not survive into later answer construction

4. Prompt Assembly / Context Contamination Seam

Class

Assembly pollution from duplicate, malformed, or carried fragments.

Signature

- repeated rules or memory cues
- malformed carried fragments
- prompt bloat
- growing instability across repeated turns

- assistant scaffolds echoed back into later turns

Owning Boundary

Prompt Assembly

Root Cause

Assembly allowed duplicate or contaminated fragments to survive and re-inflate the prompt.

Boundary Law

Prompt assembly must preserve one canonical copy of each instruction identity and reject malformed or clipped carry-forward fragments.

Fix Shape

- semantic identity enforcement for instruction blocks
- cue deduplication
- assistant-context scrubbing
- fragment validation at assembly boundary

Regression Expectations

- repeated turns remain bounded
 - malformed fragments fail closed
 - cues are unique and stable
-

5. Intent Binding / Task Canonicalization Seam

Class

Operator intuition or affect language promoted into the formal task body.

Signature

Input like:

- I feel it near OTEM
- I know it is close
- high confidence

gets turned into:

- authoritative task text
- broken work units

- false operator restatements

Owning Boundary

OTEM Task Restatement / Canonicalization

Root Cause

The system failed to distinguish actual task clauses from operator signal language.

Boundary Law

Affect, intuition, confidence, urgency, and proximity language must be treated as operator signal unless explicitly stated as the task itself.

Fix Shape

- separate task clauses from signal clauses
- canonicalize only task clauses
- preserve signal language as metadata
- do not split feeling/confidence text into work units

Regression Expectations

- feeling language remains metadata
 - task restatement contains task intent only
 - false task promotion is blocked
-

6. Gate Sizing Seam

Class

Admission gate sized for machine-shaped input instead of real operator input.

Signature

- nothing fails loudly
- human input wedges or gets squeezed
- downstream receives partial or distorted signal
- OTEM or downstream reasoning acts on fragments
- outputs look almost right, but not fully aligned

Visual Model

Big human-shaped block -> small machine-shaped gate -> wedge -> downstream starvation.

Owning Boundary

Intake / Admission Gate

Root Cause

The gate was built for small, precise, command-shaped input. Real operator input is broader, looser, clause-rich, emotional, and mixed.

Boundary Law

The admission gate must either preserve enough of the full operator task shape for downstream reasoning, or reject/clarify explicitly. It must not silently admit a distorted fragment as if it were whole.

Fix Shape

- widen intake to clause-aware parsing rather than regex grabbing
- segment clauses before canonicalization
- preserve task, signal, context, and constraint as separate meaning classes
- avoid blanket stripping that damages phrases or semantics

Regression Expectations

- mixed operator input does not silently collapse into malformed task text
- downstream reasoning receives a clean task object
- human-shaped input is preserved or explicitly rejected, not silently squeezed

7. OTEM Admission Seam

Class

False OTEM activation and contaminated OTEM task extraction.

Signature

- mere mention of OTEM triggers routing
- signal/confidence language enters canonical OTEM task text
- phrase damage from blanket token stripping, e.g. near OTEM -> near .

Owning Boundary

OTEM Admission Path

Verified Choke Path

human input -> detect_otem -> extract_otem_task -> restate_otem_task -> build_otem_result

Verified Root Cause

- detect_otem was too wide and treated mention as invocation
- extract_otem_task stripped the whole message mechanically
- restatement promoted contaminated text into authoritative operator task text
- OTEM planned on the contaminated restatement

Boundary Law

OTEM must require explicit invocation shape, not mere mention. Only task clauses may enter canonical OTEM task text.

Fix Shape

- add clause-aware OTEM analysis
- require explicit invocation shape
- split input into task_clauses and signal_clauses
- preserve operator signals as metadata
- carry metadata forward without contaminating planning

Verified Closure

- mention-only OTEM signal no longer activates routing
- mixed task/signal input is separated cleanly
- restated OTEM task excludes signal clauses
- end-to-end OTEM result keeps signal language out of the plan

Parent Insight

The session initially looked like 5-8 independent seams. The deeper architecture insight was that several downstream artifacts were not separate root causes. They were ripples from an undersized admission gate and a clause-blind OTEM path.

In other words:

- some seams were primary,
 - some were downstream manifestations,
 - and the front door mattered more than later polish.
-

Seam-Hunting Pattern Preserved From Session

Operator Observation Pattern

- feel the mismatch first
- keep the first instinct
- pressure the behavior from different angles
- do not trust pretty structure if the object underneath feels wrong
- when behavior gets cleaner, watch what still survives
- the survivor is usually closer to the root seam

Architecture Translation Pattern

- visual model: block, gate, wedge, starvation
- language model: admission seam, clause blindness, task contamination
- code model: specific functions, specific path, specific tests

This translation chain turned visual intuition into verified architecture law.

Regression Doctrine

For every seam closure, add tests for:

- mention-only and indirect human phrasing
- mixed task plus signal input
- phrase integrity after extraction or stripping
- end-to-end visible artifact correctness
- not only explicit happy-path commands

Human-shaped input must always be part of the regression set.

Operational Rule For Future Sessions

When a new seam appears, classify it by boundary first:

- admission
- binding
- assembly
- generation
- finalization
- delivery

Then ask: 1. what visible artifact proved it 2. where was the first divergence 3. what boundary owns that divergence 4. what law closes it once, not cosmetically

Closing Statement

This playbook was forged by pressure-testing, instinct, visual system modeling, code verification, and regression closure.

It preserves not just what broke, but how to detect, name, and close the next seam faster.